Python Variables & Data types

By **Prudvi**

Python Variables & Literal

Python is case sensitive

Variables:

Variables are used to store and retrieve data from our systems memory.

Literals :
Literals is basic data values for variables.

Literal Types in Python:

- *Numeric literals*
- *String literals*
- *Boolean literals*
- *Special literals*

Ex: val=10 **# val is a variable and 10 is a literal**

Python Keywords and Identifiers

Keywords

reserved words, special meanings

Cannot use as **variable names**

Python has 35 keywords

Identifiers

Identifier are name for variables, functions, class, module, etc.

Rules for Identifiers:

- Start with letter or `_`
- Followed by letters, digits, `_`
- Not a Python keyword

Comments in Python

Single-line Comments:

Use the # symbol to create a single-line comment. Anything written after the # on the same line is ignored by the interpreter.

Multi-line Comments:

- Triple-quoted Strings:

  Triple-quoted strings (''' or """) can be used as multi-line comments. Python does not treat these as true comments, but rather as string literals that are not assigned to any variable,

Ex:

```
“”” This is a multi-line comment
    using triple-quoted strings. “””
```

Docstrings in Python:

Triple quotes (“”” or ‘’’) are added right below the functions, modules, or classes to describe what they do. In Python, the docstring is then made available via the __doc__ attribute.

```
def multiply(a, b):
    """Multiplies the value of a and b"""
    return a*b

# Print the docstring of multiply function
print(multiply.__doc__)
```

Data Types

A Data Type is simply a piece of information associated with a variable to indicate to the interpreter what type of data is stored in a variable, e.g.: Number, Text etc.

- **Built in Data Types**
  - ✔ **Basic Data types** : Int, Float , Boolean, String
  - ✔ **Container types or Collections** : List, Tuple, Set, Dictionary

- **User-defined Data Types**
  Classes and Objects: You can define your own data types using classes. Instances of classes are objects.

- **Additional Data Structures (from Libraries)**
  NumPy arrays, Pandas DataFrame, Collections module deque, NamedTuples etc.

In Python, everything is an object, including int, float, bool, str, and even functions or classes themselves.

```
x = 5
print(type(x))  # Output: <class 'int'>   # An integer, instance of the `int` class
```

Mutable Vs Immutable Data Types

- **Mutable** data types: Objects whose values can be modified in place.
- **Immutable** data types: Objects whose values cannot be changed after they are created. If you want to change an immutable object, you must create a new object.

Data Type	Mutability	Example
int	Immutable	x = 10
float	Immutable	y = 3.14
bool	Immutable	is_valid = True
str	Immutable	name = "Alice"

Data Type	Mutability	Unique	Example
tuple	Immutable	No	my_tuple = (1, 2, 3) or ( )
list	Mutable	No	my_list = [1, 2, 3] or [ ]
set	Mutable	Yes	my_set = {1, 2, 3} or set()
dict	Mutable	Yes (keys)	my_dict = {"name": "Bob"} or { }

Mutable Vs Immutable

```
x = 5  # An integer object with the value 5 is created in memory, and x refers to it
print(id(x))  # Prints the memory address of the object that x refers to

x = 10  # A new integer object with the value 10 is created, and x now refers to this new object
print(id(x))  # The memory address changes because x is now pointing to a new object
```

✓ 0.0s

```
140732139588152
140732139588312
```

```
lst1=[]
dict1 = {}

for i in range(5):
    key = "key" + str(i)
    dict1[key] = i
    lst1.append(dict1)

print(lst1)
```

✓ 0.0s

```
[{'key0': 0, 'key1': 1, 'key2': 2, 'key3': 3, 'key4': 4}, {'key0': 0, 'key1':
```

```
lst2=[]

for i in range(5):
    dict2 = {}      # Create every time as it is mutable
    key = "key" + str(i)
    dict2[key] = i
    lst2.append(dict2)

print(lst2)
```

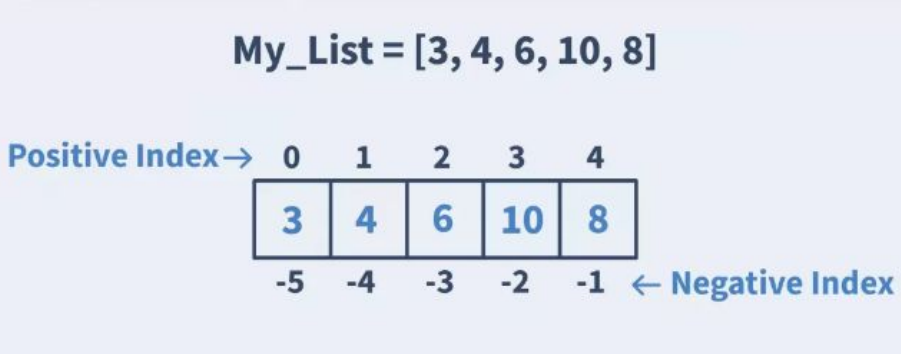
✓ 0.0s

```
[{'key0': 0}, {'key1': 1}, {'key2': 2}, {'key3': 3}, {'key4': 4}]
```

Collections - List

Key Characteristics of List

Characteristic	Description
Ordered	Maintains the order of elements added, accessible by index
Mutable	Can modify contents after creation
Allow Duplicates	Can contain multiple instances of the same element
Iterable	Can loop through elements



These indexes start from zero and are assigned in an increasing order i.e., from **zero to n – 1** where n is number of items in that lists.

Collections – List (CRUD)

 # Creating an empty List
empty_List = [ ] or list()

 # Initialize the List with Integers
integer_List = [26, 12, 97, 8]

 # Reading Lists
lst_items[0] , lst_items[-1]

 # Reading Lists – Slicing
lst_items[start:stop:step]

 # Update – Add
lst_items.append("Semiconductor")
lst_items.insert(2,"Publishing")

 # Update – change existing elements
lst_items[1]="Insurance"

#Delete
lst_items.remove("Semiconductor") ## Removing the first occurance of an item
lst_items.pop() ## Remove and return the last element
lst_items.clear() ## Remove all items from the list
Del lst_items[2] ## remove item at an index

Some of the list methods
Sort(), reverse(), index()

Collections - Tuple

Key Characteristics of Tuple

Characteristic	Description
Ordered	Tuples maintain the order of elements. The items are accessible via their index
Immutable	Once a tuple is created, its elements cannot be changed, added, or removed
Iterable	Can loop through elements

Collections – Tuple (CRUD)

```
# Creating an empty tuple
empty_tuple = ()  or tuple()
print(empty_tuple)              # Output: ()

# Creating a tuple
my_tuple = (1, 2, 3, 4, 5)
print(my_tuple)                 # Output: (1, 2, 3, 4, 5)

#Read   # Accessing elements
print(my_tuple[0])              # Output: 1 (first element)
print(my_tuple[-1])             # Output: 5 (last element)

# Read with Slicing
print(my_tuple[1:4])            # Output: (2, 3, 4)

#Update ,  # Since tuples are immutable, you cannot update them directly

#Delete,    #You cannot delete individual elements in a tuple but can delete the entire tuple
# Delete the entire tuple
  del my_tuple
# print(my_tuple)  # This would raise a NameError since my_tuple no longer exists
```

Collections - Set

Key Characteristics of Sets

Characteristic	Description
Unordered	The items in a Set do not have a specific order, and you cannot access elements by index, do not support indexing or slicing like lists or tuples. May look like auto sorted but depends on Hash and memory stored
Mutable	Can modify contents after creation
Unique Elements	Sets automatically handle duplicates, so all elements in a set are unique
Iterable	Can loop through elements
Set Operations	You can perform set operations like unions, intersections, and differences

Collections – Set (CRUD)

```
# Using set() to create an empty set as { } would create a dictionary
empty_set = set()
print(empty_set)  # Output: set()

# Creating a set
my_set = {1, 2, 3, 4, 5}
print(my_set)  # Output: {1, 2, 3, 4, 5}

# Reading elements in a set       # sets are unordered, you cannot access elements by index.
for element in my_set:
    print(element)

# Update – Add
my_set.add(6)
print(my_set)  # Output: {1, 2, 3, 4, 5, 6}

# Update  - Adding multiple elements
my_set.update([7, 8, 9])
print(my_set)  # Output: {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

Collections – Set (CRUD)

```
# Delete
my_set.remove(5)        # Raises a KeyError if the element is not found.
print(my_set)           # Output: {1, 2, 3, 4, 6, 7, 8, 9}

my_set.discard(4)       # This will not raise an error if the element is not found.
print(my_set)           # Output: {1, 2, 3, 6, 7, 8, 9}

popped_element = my_set.pop()           # Raises a KeyError if the set is empty
print(f"Popped element: {popped_element}")
print(my_set)                           # Output: (set with one fewer element)
```

Collections - Dictionaries

Key Characteristics of Dictionaries

Characteristic	Description
Unordered	Dictionaries are unordered collections, meaning that the items do not have a defined order. However, as of Python 3.7, insertion order is preserved, so items will appear in the order they were added when you iterate over them.
mutable	Dictionaries are mutable, meaning you can change their contents after creation.
Iterable	Can loop through elements

Collections – Dictionaries (CRUD)

```
# Creating empty dictionary
empty_dict = {} or dict()
print(empty_dict)  # Output: {}


# Creating a dictionary with initialization
my_dict = { 'name': 'Alice',  'age': 30,  'city': 'New York‘ }
print(my_dict)              # Output: {'name': 'Alice', 'age': 30, 'city': 'New York'}

# Using dict() to create a dictionary
another_dict = dict(name='Bob', age=25)
print(another_dict)       # Output: {'name': 'Bob', 'age': 25}

# Read  # Accessing values
print(my_dict['name'])  # Output: 'Alice'

# Read Using get() method
print(my_dict.get('age'))  # Output: 30
print(my_dict.get('country', 'Not Found'))  # Output: 'Not Found' (default value)
```

Collections – Dictionaries (CRUD)

```
# Adding a new key-value pair
my_dict['height'] = 5.5
print(my_dict)          # Output: {'name': 'Alice', 'age': 31, 'city': 'New York', 'height': 5.5}

# Updating a value
my_dict['age'] = 31
print(my_dict)          # Output: {'name': 'Alice', 'age': 31, 'city': 'New York'}

# Deleting a key-value pair
del my_dict['city']
print(my_dict)          # Output: {'name': 'Alice', 'age': 31, 'height': 5.5}

# Removing a key using pop()
age = my_dict.pop('age')
print(age)          # Output: 31
print(my_dict)          # Output: {'name': 'Alice', 'height': 5.5}
```

Collections – Dictionaries (CRUD)

```
# Iterating over keys
for key in my_dict:
        print(key)

# Iterating over values
for value in my_dict.values():
    print(value)

# Iterating over key-value pairs
for key, value in my_dict.items():
    print(f"{key}: {value}")
```

All Collection Operations

Operation	List	Tuple	Set	Dictionary
1. Empty Initialization	[] list()	() tuple()	set() (Note: {} creates a dict)	{} dict()
2. Initialization	[1, 2, 3], list([1, 2, 3])	(1, 2, 3), tuple([1, 2, 3])	{1, 2, 3}, set([1, 2, 3])	{'a': 1, 'b': 2, 'c': 3}, dict([('a', 1), ('b', 2)])
3. Read (using index or slicing)	lst[0] lst[1:3]	tpl[0] tpl[1:3]	Cannot access by index; use iteration	dct['a'] dct.get('a')
4. Update by Index	lst[0] = 10	Not supported (immutable)	Not supported (unordered)	dct['a'] = 10 #using keys
5. Insert at Index	lst.insert(1, 100)	Not supported (immutable)	Not supported (unordered)	Not supported (unordered)
6. Append/Add	lst.append(56)	Not supported (immutable)	st.add(56)	dct['d'] = 56 #using keys
7. Delete by Value	lst.remove(94) lst.clear()	Not supported (immutable)	st.remove(94) st.discard(94) st.clear()	dct.pop('b') dct.popitem() dct.clear()

8. Delete by Index	del lst[0]
	lst.pop(0)
		Not supported (immutable)	Not supported (no indexing)	del dct['a'] # using keys
				dct.pop('a')

Collections - Comprehension

Collection comprehensions in Python provide a concise way to create collections (like lists, sets, and dictionaries) based on existing iterables.

Tuples do not have a dedicated comprehension syntax like lists, sets, or dictionaries.

Syntax : [expression for item in iterable] or [expression for item in iterable if condition]

List Comprehension

```
squares = [x**2 for x in range(10)]
print(squares)  # Output: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Set Comprehension

```
squares_set = {x**2 for x in range(10)}
print(squares_set)  # Output: {0, 1, 4, 9, 16, 25, 36, 49, 64, 81}
```

Dictionary Comprehension

```
squares_dict = {x: x**2 for x in range(5)}
print(squares_dict)  # Output: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Tuple Comprehension Equivalent

While you can't directly create a tuple using a comprehension, you can use a **generator expression** enclosed in the tuple() function.

```
squares_tuple = tuple(x**2 for x in range(10))
print(squares_tuple)  # Output: (0, 1, 4, 9, 16, 25, 36, 49, 64, 81)
```

Generator expression & object

Generator objects are different from collections such as list, set, dictionary, Let see how ?

Syntax: generator = (expression for item in iterable)

```
    ex:  squares_gen = (x**2 for x in range(10))
# to access values
for square in squares_gen:
        print(square)
```

Key Characteristics of Generator Objects

- **Lazy Evaluation**: The values are generated on-the-fly and are not computed until you iterate over the generator.
- **Memory Efficient**: Since it doesn't store all the values in memory, it's suitable for large data sets or streams.
- **Single Iteration**: You can iterate through a generator only once. After that, it's exhausted, and you would need to create a new generator to iterate again.

Thank you